

Business Requirements Document (BRD)

Overview

The purpose of this document is to outline the requirements for a new API feature that will determine if a given string is encrypted or decrypted. Based on the identification, the API will either decrypt an encrypted string or encrypt a decrypted string. This feature aims to enhance the usability of the existing encryption and decryption functionalities by providing a seamless experience for users.

Existing System Summary

The current system includes the following APIs:

- **GET /test**: A simple endpoint to verify that the API is operational.
- **GET /enc**: An endpoint that encrypts a given string based on a provided secret key.
- **GET /dec**: An endpoint that decrypts a given encrypted string based on a provided secret key.

These endpoints are implemented in the `MainController`` class, utilizing the `DynamicIV_Encrypt_Decrypt`` resource for encryption and decryption processes.

Proposed Changes

To implement the new feature, the following changes are proposed:

1. **New Endpoint**: Introduce a new API endpoint:
 - **GET /identify**: This endpoint will accept a string and determine if it is encrypted or decrypted.
 - If the string is encrypted, it will return the decrypted version.
 - If the string is decrypted, it will return the encrypted version.
2. **Logic Implementation**: Develop logic to identify whether the input string is encrypted or decrypted. This may involve checking the string format, length, or specific patterns that indicate encryption.

Business Rules

1. The API must return an error message if the input string is null or empty.
2. The API must validate the provided secret key and return an error if it is invalid.
3. The API should handle exceptions gracefully and return appropriate HTTP status codes (e.g., 400 for bad requests, 500 for server errors).
4. The response format should be consistent with existing API responses (JSON format).

APIs to Consider

- **GET /identify**: New endpoint for identifying and processing strings.
- Existing endpoints (`/test``, `/enc``, `/dec``) will remain unchanged but may need to be updated for integration with the new endpoint.

Risks & Assumptions

Risks

- **Security Risks**: Improper handling of encryption keys may expose sensitive data.
- **Performance Risks**: The logic for identifying encrypted vs. decrypted strings may introduce latency.
- **Compatibility Risks**: Changes to the API may affect existing clients relying on current functionality.

Assumptions

- The existing encryption and decryption methods are reliable and secure.
- Users have a basic understanding of how to interact with the API and provide the necessary parameters.

Next Steps

1. **Design**: Create a detailed design document for the new `/identify`` endpoint, including input/output specifications.
2. **Development**: Implement the new endpoint and the logic for string identification.
3. **Testing**: Conduct unit and integration testing to ensure the new feature works as intended and does not break existing functionality.
4. **Documentation**: Update API documentation to include the new endpoint and its usage.
5. **Deployment**: Plan for deployment in a staging environment for further testing before going live.

By following these steps, we aim to enhance the API's functionality while ensuring a smooth user experience.